

15. Roadmap de Desarrollo y Evolución de OmniMerge

1. Introducción

OmniMerge ha evolucionado considerablemente desde su planteamiento inicial.

El objetivo original del proyecto consistía en permitir la creación de entidades o competidores que pudieran participar en torneos hasta determinar uno o varios ganadores. Sin embargo, durante el desarrollo se identificó que limitar el sistema exclusivamente a torneos impediría aprovechar muchas de las estructuras necesarias para representar competidores complejos.

Por esta razón, OmniMerge comenzó a evolucionar hacia una plataforma mucho más general.

Actualmente el proyecto comprende, entre otros elementos:

- Entidades genéricas.
- Tipos de entidades.
- Atributos dinámicos.
- Opciones de atributos.
- Relaciones entre atributos.
- Relaciones entre opciones.
- Condiciones de atributos.
- Grupos de atributos.
- Colecciones.
- Versiones de entidades.
- Biblioteca reutilizable.
- Contenido público y privado.
- Comunidad.
- Plantillas de fases.
- Motores de competición.
- Puertas de entrada.
- Puertas de salida.
- Reglas de clasificación.
- Tournament Graph.
- Bifurcaciones y uniones entre fases.
- Competition Lab.
- Ejecución de encuentros.
- Series de encuentros.
- Decisiones manuales.
- Sistemas de eliminación.
- Sistemas de clasificación.

El desarrollo del subsistema de torneos ha alcanzado un punto especialmente importante.

El motor **Single Elimination** ya dejó de ser únicamente una configuración visual y comenzó a convertirse en un sistema realmente ejecutable. Esto representa un cambio de etapa para OmniMerge.

A partir de este punto, el objetivo ya no debe ser continuar agregando funcionalidades de forma aislada, sino completar sistemáticamente todos los componentes necesarios para convertir OmniMerge en una plataforma completa de competición, simulación y gestión de universos.

Este documento define esa ruta.

2. Estado actual del proyecto

2.1. Estado general

A agosto de 2026, OmniMerge ya cuenta con una arquitectura considerablemente más avanzada que un CRUD tradicional.

La aplicación utiliza principalmente:

- Laravel 12.
- PHP 8.2.
- MySQL.
- Blade.
- Tailwind CSS.
- Alpine.js.
- Vite.

La arquitectura del backend está organizada mediante:

- Models.
- Controllers.
- Form Requests.
- Policies.
- Services.
- Actions.
- Motores especializados.
- Validadores.
- Presenters.
- Runtime services.

El frontend se construye principalmente mediante Blade, Tailwind y Alpine, manteniendo una interfaz administrativa consistente entre los diferentes módulos.

3. Estado actual del sistema de torneos

El sistema de torneos actualmente puede dividirse conceptualmente en cuatro grandes capas.

Plantillas de fases



Tournament Graph



Competition Lab



Motores de ejecución

Los motores principales contemplados actualmente son:

Single Elimination

Round Robin

Group Stage

Swiss

Por el momento:

LEAGUE

CUSTOM

deben mantenerse fuera del flujo ejecutable hasta disponer de motores reales que los soporten.

4. Situación actual de Single Elimination

Single Elimination se encuentra actualmente en la posición más avanzada de los motores de OmniMerge.

Ya no debe considerarse solamente como una funcionalidad más.

A partir de este punto debe convertirse en el:

motor de referencia de OmniMerge.

Esto significa que su comportamiento servirá para establecer el contrato que posteriormente deberán cumplir Round Robin, Group Stage, Swiss y cualquier nuevo motor.

Single Elimination ya ha avanzado en elementos como:

- generación de estructura;
- rondas;
- encuentros;
- participantes;
- progresión;
- BYE;
- seeding;
- pairing;
- selección de clasificados;
- resultados;
- series;
- puertas;
- conexiones;
- clasificación;
- eliminación;
- integración con Competition Lab;
- interacción con Tournament Graph;
- decisiones manuales;
- estructuras avanzadas.

Por lo tanto, el siguiente objetivo no consiste en continuar expandiéndolo indefinidamente.

Primero debe cerrarse formalmente como **versión funcional de referencia**.

5. Decisión arquitectónica principal

La estrategia de desarrollo a partir de este punto será:

Completar un motor correctamente



Definir su contrato



Usarlo como referencia



Completar el siguiente motor



Completar Tournament Graph



Crear torneos persistentes



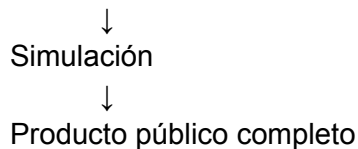
Conectar entidades reales



Guardar historial



Universos



Esto evita uno de los mayores riesgos del proyecto:

que existan muchos módulos parcialmente implementados pero ninguno completamente funcional.

6. Roadmap general

La nueva ruta de desarrollo queda organizada de la siguiente manera:

- P0 Seguridad funcional y eliminación de engaños
 - P1 Paridad Preview / Runtime
 - P2 Funcionalidades avanzadas de competición

 - P3 Cierre definitivo de Single Elimination
 - P4 Round Robin completo
 - P5 Group Stage completo
 - P6 Swiss completo
 - P7 Tournament Graph completamente ejecutable
 - P8 Runtime persistente de torneos
 - P9 Torneos reales con entidades reales
 - P10 Historial, estadísticas y resultados
 - P11 Consolidación de Biblioteca y Comunidad
 - P12 Universos
 - P13 Motor de simulaciones e interacciones
 - P14 Plataforma pública y colaboración
 - P15 Calidad, seguridad y preparación para producción
-

7. P0 — Seguridad funcional y eliminación de funcionalidades engañosas

Objetivo

Eliminar cualquier parte de la interfaz que indicara al usuario que una función estaba disponible cuando realmente todavía no podía ejecutarse correctamente.

Esta prioridad tuvo como principio:

La interfaz nunca debe prometer algo que el backend todavía no puede cumplir.

Trabajo realizado

Entre los problemas tratados se encontraron:

- botones de edición sin funcionamiento real;
- funcionalidades Swiss incompletas;
- modos League y Custom visibles sin motor;
- acciones disponibles en vistas públicas sin autorización;
- clonación que no respetaba `allow_cloning`;
- conteos de fases utilizando estructuras antiguas;
- coexistencia del sistema antiguo de fases con Tournament Graph;
- ejecución de grafos con motores inexistentes;
- configuraciones Best of todavía no soportadas.

Resultado esperado

La aplicación debe diferenciar claramente entre:

Disponible

Próximamente

No permitido

No soportado

P0 representa la base de confianza entre sistema y usuario.

8. P1 — Paridad entre Preview y Runtime

Objetivo

Evitar que el diseñador visual mostrara un comportamiento diferente al que posteriormente ejecutaba Competition Lab.

Antes de esta prioridad existía el riesgo de que:

Preview dice A

Runtime hace B

Esto era especialmente peligroso en:

- merges;
- conexiones;
- participantes eliminados;
- reglas por ronda;
- clasificación;
- Swiss;
- grafos avanzados.

Principio establecido

A partir de esta prioridad:

Preview y Runtime deben compartir las mismas reglas siempre que sea posible.

Elementos trabajados

Se avanzó en:

- políticas de merge;
 - participantes eliminados por ronda;
 - `ON_ELIMINATION`;
 - grafos avanzados de Single Elimination;
 - calculador compartido de Swiss;
 - routing durante ejecución;
 - compatibilidad entre Tournament Graph y motores.
-

9. P2 — Funcionalidades avanzadas de competición

Objetivo

Eliminar varias limitaciones que todavía impedían representar competiciones reales.

P2 introdujo conceptos transversales que no pertenecen exclusivamente a un motor.

Entre ellos:

Decisiones manuales

Se estableció el concepto:

RUNNING



AWAITING_DECISION



usuario resuelve



RUNNING

Esto permite implementar correctamente situaciones como:

- asignación manual de grupos;
 - selección de BYE;
 - orden manual;
 - seed manual;
 - resolución manual de empates;
 - decisiones Swiss.
-

Series Best of

Se creó una capa compartida para representar:

BO1

BO3

BO5

BO7

BO9

FIXED_GAMES

Un encuentro ya no necesariamente equivale a un único juego.

Conceptualmente:

Match

├── Game 1

├── Game 2

├── Game 3

├── ...

└── Resultado de la serie

Políticas de empate

Se contemplaron diferentes estrategias:

USE_TIEBREAKERS

RANDOM_RESOLUTION

INCLUDE_ALL_TIED
MANUAL_RESOLUTION
REQUIRE_PLAYOFF

Esto permite diferenciar correctamente:

- orden visual;
 - empate competitivo;
 - resolución del último cupo.
-

10. P3 — Cierre definitivo de Single Elimination

Estado

Single Elimination ya posee vida propia dentro del sistema y puede considerarse el motor más avanzado de OmniMerge.

Sin embargo, antes de iniciar el desarrollo completo de los demás motores es necesario realizar una última etapa de cierre.

Objetivo de P3

Declarar oficialmente:

Single Elimination V1 está terminado y será el motor de referencia de OmniMerge.

A partir de ese momento no se agregarán funcionalidades arbitrariamente.

Únicamente:

- correcciones;
 - optimizaciones;
 - mejoras compatibles;
 - funcionalidades transversales.
-

P3.1. Auditoría de configuración

Debe verificarse cada parámetro disponible.

Por ejemplo:

Seeding

INPUT
RANKING
RANDOM
MANUAL

Debe comprobarse que todos tengan un efecto real.

P3.2. Pairing

Verificar estrategias como:

SEQUENTIAL
RANDOM
STANDARD_SEEDED

y garantizar que el bracket generado corresponda realmente a cada estrategia.

11. P3.3. BYE

Debe comprobarse especialmente con cantidades irregulares.

Ejemplos:

3 participantes
5 participantes
6 participantes
7 participantes
10 participantes
13 participantes

Debe utilizarse correctamente:

siguiente potencia de 2
-
cantidad de participantes
=
cantidad de BYEs

Ejemplo:

5 participantes
siguiente potencia = 8

$8 - 5 = 3$ BYEs

12. P3.4. Series

Probar:

BO1
BO3
BO5
BO7
BO9
FIXED_GAMES

Debe comprobarse:

- cuándo termina una serie;
 - cuándo continúa;
 - qué ocurre con empates;
 - qué ocurre cuando se requiere ganador;
 - cómo se muestran resultados parciales.
-

13. P3.5. Clasificados múltiples

Single Elimination no debe limitarse obligatoriamente a:

2 participantes → 1 ganador

Debe poder representar estructuras:

K participantes → Q clasificados

cuando la configuración de encuentro lo permita.

14. P3.6. Estructuras avanzadas

Verificar completamente:

- rounds;
- encounters;
- slots;
- input gates;
- results;
- connections;
- routing.

El visualizador y el runtime deben representar la misma estructura.

15. P3.7. Puertas de salida

Probar casos como:

ganador
subcampeón
semifinalistas
eliminados en ronda 1
eliminados en ronda 2
TOP N
posición específica

16. P3.8. Pruebas automáticas

Single Elimination debe convertirse en una referencia protegida mediante tests.

Deberían existir pruebas para:

- generación;
 - BYE;
 - seeding;
 - pairing;
 - series;
 - decisiones manuales;
 - clasificación;
 - eliminación;
 - puertas;
 - routing;
 - estructuras avanzadas.
-

17. Resultado de P3

Cuando P3 termine deberá existir una **Definition of Done** clara.

Single Elimination pasará entonces de:

motor en desarrollo

a:

motor estable de referencia

18. P4 — Round Robin completo

Objetivo

Convertir Round Robin en el segundo motor completamente ejecutable.

No se debe desarrollar únicamente su formulario.

Debe poder jugarse una competición completa.

19. P4.1. Calendario

Implementar correctamente la generación automática de enfrentamientos.

Ejemplo:

A
B
C
D

debe producir:

Ronda 1

A vs D

B vs C

Ronda 2

A vs C

D vs B

Ronda 3
A vs B
C vs D

20. P4.2. Ida y vuelta

Debe contemplarse:

single round robin
double round robin

y posteriormente permitir configuraciones mayores si existe una necesidad real.

21. P4.3. Sistema de puntuación

Configuraciones como:

victoria = 3
empate = 1
derrota = 0

no deben estar codificadas rígidamente.

La puntuación debe formar parte de la configuración.

22. P4.4. Clasificación en vivo

Después de cada encuentro deben actualizarse:

PJ
PG
PE
PP
GF
GC
DG
Puntos

o sus equivalentes genéricos cuando la competición no represente deportes tradicionales.

23. P4.5. Tiebreakers

Implementar cadenas ordenadas.

Ejemplo:

1. puntos
 2. enfrentamiento directo
 3. diferencia
 4. score a favor
 5. criterio adicional
-

24. P4.6. Clasificación y eliminación

Round Robin debe poder emitir:

TOP N
BOTTOM N
POSITION
RANGE

hacia Tournament Graph.

25. P4.7. Empate en el corte

Caso:

clasifican 4

pero posiciones:

- 3.º = 10 pts
- 4.º = 8 pts
- 5.º = 8 pts
- 6.º = 8 pts

El sistema debe aplicar explícitamente la política configurada.

26. Resultado de P4

Debe ser posible realizar:

crear Round Robin

- agregar participantes
 - generar calendario
 - registrar resultados
 - actualizar clasificación
 - resolver empates
 - obtener clasificados
 - enviar clasificados al Graph
-

27. P5 — Group Stage completo

Group Stage debe construirse utilizando Round Robin como base conceptual.

Un grupo debe comportarse esencialmente como una competición independiente.

28. P5.1. Distribución de participantes

Implementar y validar:

BALANCED
SNAKE
RANDOM
RANKING
POTS
MANUAL

29. P5.2. Capacidades

Permitir grupos con capacidades diferentes.

Ejemplo:

Grupo A → 5
Grupo B → 5
Grupo C → 4
Grupo D → 4

30. P5.3. Clasificación por grupo

Cada grupo debe tener su propia tabla.

Grupo A

- 1.
- 2.
- 3.
- 4.

Grupo B

- 1.
 - 2.
 - 3.
 - 4.
-

31. P5.4. Reglas de avance

Casos como:

Top 2 de cada grupo
Mejor tercero
Mejores 4 terceros
Ganadores de grupo
Últimos de cada grupo
Posiciones 2-3

32. P5.5. Comparaciones entre grupos

Uno de los problemas más importantes.

Ejemplo:

Grupo A tercero
Grupo B tercero
Grupo C tercero

Grupo D tercero

Debe existir una clasificación secundaria para decidir cuáles avanzan.

33. P5.6. Normalización

Debe contemplarse el caso donde los grupos tengan diferente cantidad de participantes.

No siempre será válido comparar puntuaciones brutas.

34. Caso de referencia

El escenario principal para validar Group Stage debería ser similar a:

Fase de grupos



1.º y 2.º avanzan automáticamente



terceros compiten por plazas adicionales



clasificados enviados a eliminación directa

35. P6 — Swiss completo

Objetivo

Convertir Swiss en un sistema dinámico real.

Swiss es más complejo porque cada ronda depende del estado generado por las anteriores.

36. P6.1. Emparejamiento dinámico

Debe considerar:

- score;

- récord;
 - ranking;
 - encuentros anteriores;
 - participantes disponibles.
-

37. P6.2. Evitar rematches

Debe evitarse enfrentar nuevamente a dos participantes siempre que exista una alternativa válida.

38. P6.3. Floaters

Cuando un grupo de puntuación sea impar, algunos participantes deberán poder subir o bajar de grupo.

39. P6.4. BYE Swiss

Debe existir control sobre:

- quién puede recibir BYE;
 - cuántos ha recibido;
 - prioridad;
 - elección automática;
 - elección manual.
-

40. P6.5. Clasificación por récord

Ejemplo:

4-0
3-1
3-1
2-2
1-3

El sistema debe ser capaz de tomar decisiones durante la competición utilizando esos récords.

41. P6.6. Clasificación anticipada

Ejemplo:

3 victorias → clasifica
3 derrotas → eliminado

Un participante podría abandonar el sistema Swiss antes del final total de rondas.

42. P6.7. Tiebreakers Swiss

Contemplar:

- opponent score;
 - head-to-head;
 - Buchholz u otras variantes cuando correspondan;
 - eliminación del peor resultado del oponente;
 - criterios configurables.
-

43. Resultado P6

El sistema debería ser capaz de ejecutar:

16 participantes
→ ronda 1
→ resultados
→ emparejamiento ronda 2
→ resultados
→ ...
→ clasificación/eliminación
→ resultado final

sin intervención técnica.

44. P7 — Tournament Graph completamente ejecutable

Esta etapa integra todos los motores anteriores.

Hasta aquí se prueba una fase.

Desde P7 se prueba el torneo completo.

45. P7.1. Flujo lineal

Ejemplo:

Inicio
↓
Group Stage
↓
Single Elimination
↓
Campeón

46. P7.2. Bifurcaciones

Ejemplo:

┌→ Clasificados
Fase A ———┤
└→ Eliminados

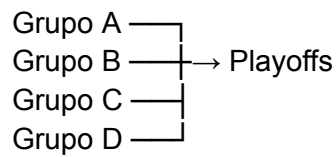
47. P7.3. Convergencias

Ejemplo:

Clasificatoria A ———┐
└→ Fase Final
Clasificatoria B ———┐

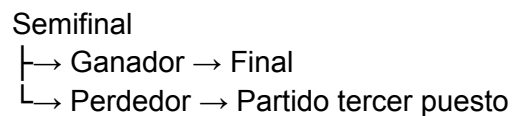
48. P7.4. Múltiples ramas

Ejemplo:



49. P7.5. Tercer puesto

Ejemplo:



50. P7.6. Múltiples terminales

No todo Tournament Graph necesita un único ganador.

Podrían existir:

- Champion
- Runner-up
- Third place
- Qualified
- Eliminated
- Secondary winner

51. P7.7. Merge policies

Validar completamente políticas como:

- WAIT_ALL
- APPEND
- FIRST_AVAILABLE
- PRIORITY

52. Resultado P7

Competition Lab deberá poder ejecutar grafos complejos de principio a fin.

53. P8 — Runtime persistente de torneos

Esta será una de las etapas arquitectónicas más importantes.

Actualmente Competition Lab funciona como laboratorio temporal.

Eso es correcto.

Debe continuar existiendo.

Pero un torneo real necesita persistencia.

54. Diferencia entre Lab y Runtime

Competition Lab

Será usado para:

- probar
- experimentar
- validar
- simular estructuras
- verificar Tournament Graph

Su estado puede continuar siendo temporal.

Tournament Runtime

Será usado para:

- torneos reales
- resultados oficiales
- historial
- continuación posterior

55. P8.1. Persistencia

El torneo deberá almacenar:

- estado;
 - participantes;
 - nodos;
 - encounters;
 - juegos;
 - resultados;
 - decisiones;
 - clasificaciones;
 - eventos.
-

56. P8.2. Reanudación

Debe ser posible:

crear torneo

→ jugar 3 partidas

→ cerrar aplicación

→ regresar mañana

→ continuar exactamente desde ese punto

57. P8.3. Snapshot

Cuando comienza un torneo debe congelarse la configuración utilizada.

Por ejemplo:

Tournament Template V3

Si posteriormente alguien edita la plantilla y crea:

Tournament Template V4

el torneo antiguo debe continuar utilizando V3.

58. P8.4. Historial de eventos

Conviene desarrollar una secuencia semejante a:

TOURNAMENT_CREATED
PARTICIPANT_ADDED
TOURNAMENT_STARTED
PHASE_STARTED
MATCH_STARTED
GAME_RECORDED
MATCH_COMPLETED
PARTICIPANT_QUALIFIED
PARTICIPANT_ELIMINATED
PHASE_COMPLETED
TOURNAMENT_COMPLETED

59. P9 — Torneos reales

Una vez exista Runtime persistente se conectará con las entidades reales de OmniMerge.

60. Flujo esperado

Crear torneo
↓
Elegir Tournament Template
↓
Seleccionar participantes
↓
Validar compatibilidad
↓
Configurar seed
↓
Resolver decisiones iniciales
↓
Iniciar
↓
Ejecutar encuentros
↓
Mover participantes por Tournament Graph

↓
Finalizar

61. Selección de participantes

Los participantes podrán provenir de:

- entidades;
 - colecciones;
 - versiones;
 - búsqueda;
 - selección manual;
 - futuras reglas de universo.
-

62. P10 — Historial y estadísticas

Un torneo terminado debe convertirse en información reutilizable.

63. Historial del participante

Ejemplo:

Naruto Uzumaki

Torneos: 14
Campeonatos: 3
Subcampeonatos: 2
Partidas: 74
Victorias: 51
Derrotas: 18
Empates: 5

64. Historial del torneo

Debe conservar:

- participantes;
 - bracket;
 - fases;
 - resultados;
 - rutas;
 - clasificaciones;
 - eliminaciones;
 - campeón.
-

65. Estadísticas

Posteriormente podrán existir:

Win rate

Participaciones

Campeonatos

Rondas alcanzadas

Promedio de posición

Rivales frecuentes

Historial directo

66. P11 — Consolidación de Biblioteca

Una vez que los torneos funcionen completamente, será necesario volver a realizar una auditoría general de Biblioteca.

67. Elementos

Revisar:

- Entity Types.
- Entities.
- Attributes.
- Attribute Options.
- Attribute Groups.
- Relationships.
- Conditions.
- Collections.
- Versions.

- Políticas.
 - Cloning.
 - Community.
 - Hub.
-

68. Objetivo

Conseguir el flujo:

Usuario A crea contenido
↓
lo publica
↓
Usuario B lo encuentra
↓
revisa
↓
clona/reutiliza si está permitido
↓
lo utiliza en su propio contexto

69. P12 — Universos

Esta etapa representa uno de los mayores objetivos conceptuales de OmniMerge.

Un Universo no debe ser simplemente una carpeta.

Debe establecer contexto.

70. Un Universo podrá contener

- tipos de entidades;
- entidades;
- versiones;
- atributos;
- colecciones;
- reglas;
- torneos;
- relaciones;

- simulaciones.
-

71. Ejemplo

Universo Naruto

- Personajes
- Aldeas
- Clanes
- Equipos
- Bijū
- Técnicas
- Versiones
- Colecciones
- Torneos
- Simulaciones

72. Versiones dentro de universos

Ejemplo:

Naruto niño
Naruto Shippuden
Naruto Modo Sabio
Naruto Kurama
Naruto Hokage

podrían representar diferentes estados/versiones de una misma entidad base.

73. P13 — Motor de simulación

Hasta ese punto los resultados podrán introducirse manualmente.

P13 permitirá determinar resultados automáticamente.

74. Result Provider

El Tournament Engine no debería calcular directamente quién gana.

Conceptualmente debería preguntar:

ResultProvider



resultado

75. Posibles proveedores

ManualResultProvider

RandomResultProvider

AttributeResultProvider

FormulaResultProvider

RuleResultProvider

SimulationResultProvider

AIResultProvider

76. Ejemplo

Un torneo podría configurarse para utilizar:

60 % poder

20 % velocidad

10 % inteligencia

10 % resistencia

y producir resultados utilizando los atributos de las entidades.

Posteriormente podrían existir motores mucho más complejos.

77. Ventaja arquitectónica

Tournament Graph no necesitaría conocer cómo se determina el resultado.

Únicamente recibiría:

resultado

ganador

perdedor

estadísticas

Por lo tanto podrían reutilizarse exactamente los mismos motores de torneo.

78. P14 — Plataforma pública y colaboración

Después de consolidar las funcionalidades internas se desarrollará con mayor profundidad la capa social.

79. Funciones futuras

- perfiles públicos;
 - contenido público;
 - búsqueda global;
 - compartir colecciones;
 - compartir atributos;
 - compartir Tournament Templates;
 - clonar;
 - fork;
 - créditos;
 - autoría;
 - versiones públicas;
 - comentarios o valoraciones futuras;
 - rankings de contenido;
 - torneos públicos.
-

80. Tournament Viewer público

Un torneo podría disponer de una página pública con:

Participantes
Bracket
Fases
Resultados
Clasificación
Encuentros
Historial
Campeón

81. P15 — Preparación para producción

P15 representa el cierre técnico antes de considerar OmniMerge una plataforma estable.

82. Seguridad

Auditar:

- Policies.
 - Authorization.
 - CSRF.
 - Mass assignment.
 - XSS.
 - Uploads.
 - permisos.
 - visibilidad.
 - clonación.
 - información privada.
-

83. Integridad

Revisar:

- transacciones;
 - concurrencia;
 - resultados duplicados;
 - race conditions;
 - estados inválidos;
 - rollback;
 - idempotencia.
-

84. Base de datos

Optimizar:

- índices;
 - relaciones;
 - foreign keys;
 - consultas;
 - eager loading;
 - N+1;
 - almacenamiento histórico.
-

85. Testing

Construir una pirámide de pruebas.

Unit Tests



Feature Tests



Integration Tests



Tournament Engine Tests



Graph Tests



End-to-End Tests

86. CI/CD

Posteriormente:

GitHub



tests



static analysis



frontend build



deployment

87. Observabilidad

Agregar:

- logs estructurados;
 - errores del runtime;
 - errores de motores;
 - eventos;
 - auditoría administrativa.
-

88. Documentación final

El proyecto deberá terminar contando con documentación de:

Arquitectura
Backend
Frontend
Dominio
Biblioteca
Tournament Engine
Tournament Graph
Runtime
Universos
Simulation Engine
API
Deployment
Testing

89. Funcionalidades que NO deben priorizarse todavía

Existen muchas funciones atractivas que podrían desarrollarse inmediatamente, pero hacerlo ahora desviaría el proyecto.

Por ejemplo:

League Engine
Custom Engine
IA avanzada
chat social
gamificación
rankings públicos
aplicación móvil

API pública completa

Estas funciones deberán esperar.

Actualmente es más importante completar el núcleo.

90. Orden de dependencias

La razón del orden definido anteriormente es que las prioridades dependen entre sí.

Single Elimination



Round Robin



Group Stage



Swiss



Tournament Graph



Persistent Runtime



Real Tournament



History



Library Integration



Universes



Simulation

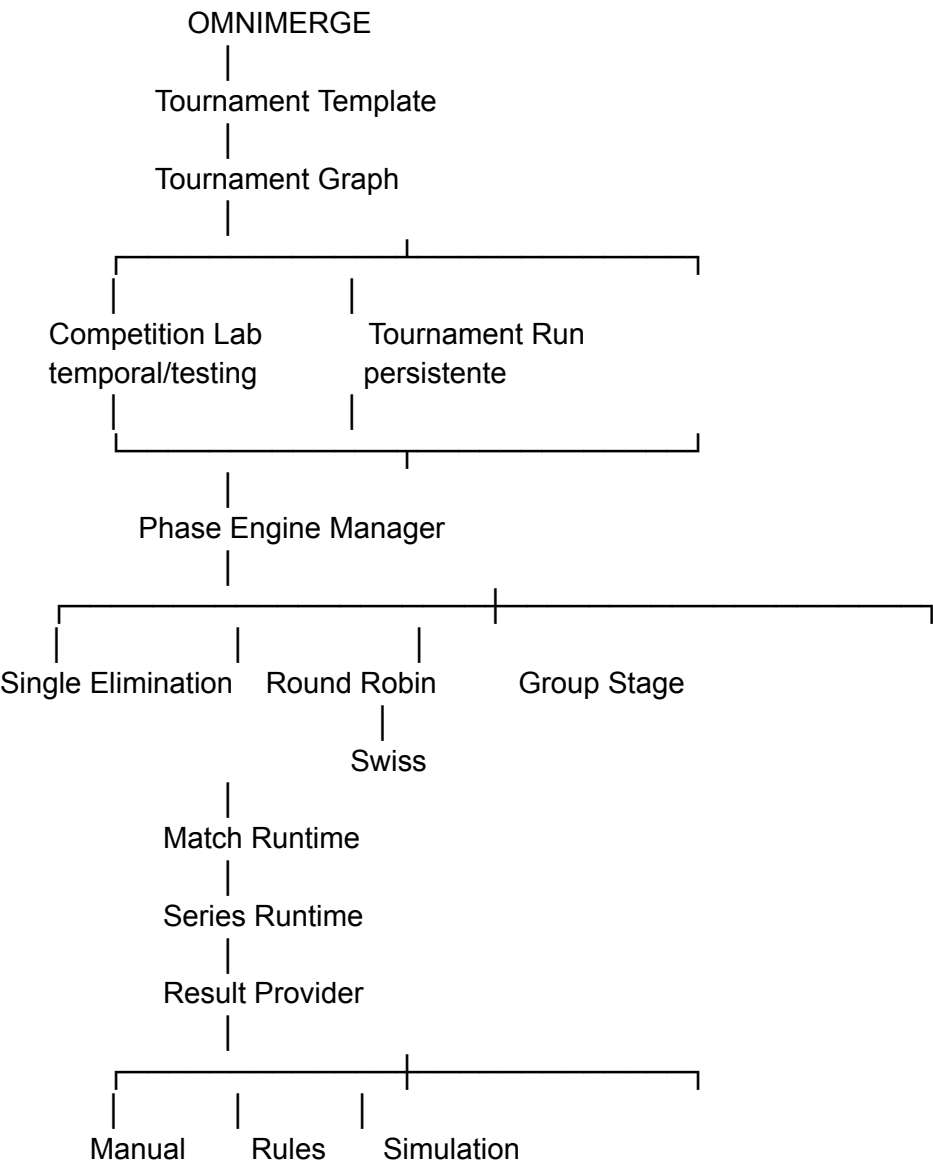


Public Platform

No todas las dependencias son estrictamente técnicas, pero este orden reduce considerablemente el riesgo de reescrituras.

91. Arquitectura objetivo del sistema de torneos

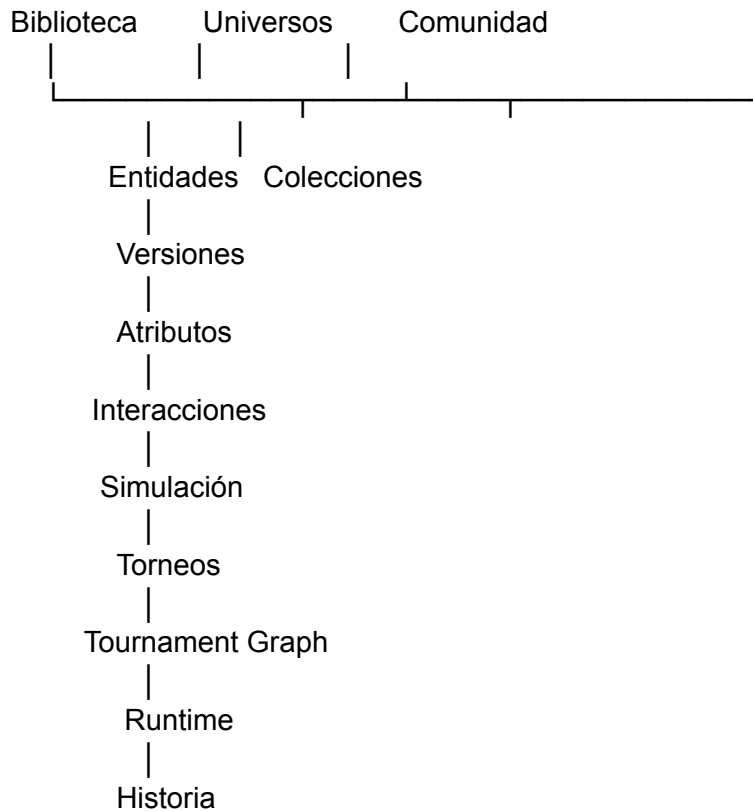
Al finalizar estas etapas, la arquitectura conceptual debería aproximarse a:



92. Arquitectura objetivo global de OmniMerge

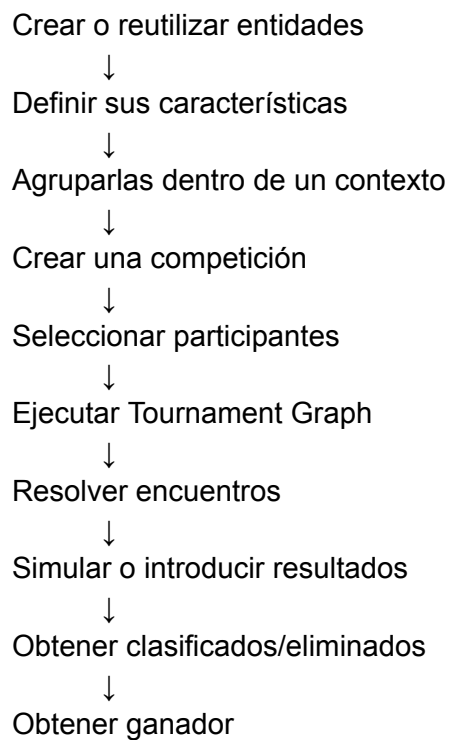
El objetivo final es conseguir aproximadamente:





93. Objetivo funcional final

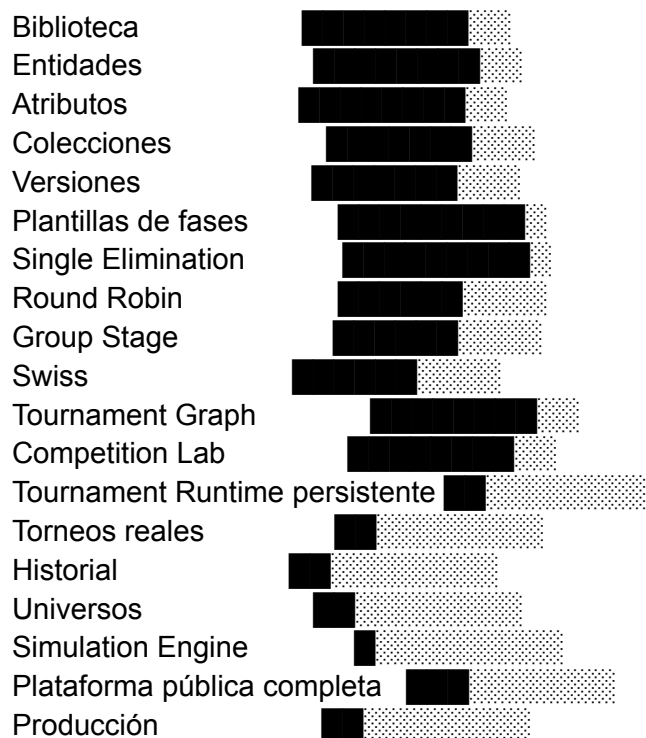
El flujo ideal de OmniMerge será:



↓
Guardar historia
↓
Reutilizar los resultados

94. Estado actual respecto al objetivo

Puede representarse de la siguiente manera:



Estas barras no representan porcentajes matemáticos exactos.

Su finalidad es mostrar visualmente el estado relativo de madurez de los subsistemas.

95. Próxima acción inmediata

La siguiente prioridad será:

P3 — Cierre de Single Elimination

No se comenzará inmediatamente a crear nuevas funcionalidades grandes.

Primero se realizará una auditoría completa de Single Elimination.

La auditoría deberá producir una matriz semejante a:

Funcionalidad	Estado	Pruebas	Acción
Seeding INPUT	Completo	Sí/No	—
Seeding RANDOM	Completo	Sí/No	—
Seeding MANUAL	Revisar	Sí/No	Corregir
BYE automático	Completo	Sí/No	—
BYE manual	Revisar	Sí/No	Validar
BO1	Completo	Sí/No	—
BO3	Revisar	Sí/No	Validar
BO5	Revisar	Sí/No	Validar
K→Q	Revisar	Sí/No	Validar
Exits	Revisar	Sí/No	Validar
ON_ELIMINATION	Revisar	Sí/No	Validar
Graph avanzado	Revisar	Sí/No	Validar
Decisiones manuales	Revisar	Sí/No	Validar
Estructura interna	Revisar	Sí/No	Validar

Cuando todos los elementos críticos estén en:

COMPLETO

+

PROBADO

Single Elimination será declarado:

ENGINE STABLE V1

96. Regla para las siguientes prioridades

A partir de P3 se aplicará una regla estricta:

Una prioridad no se considerará terminada porque la interfaz exista. Se considerará terminada cuando configuración, dominio, runtime, frontend y pruebas representen el mismo comportamiento.

Por lo tanto:

UI
+
Backend
+
Runtime
+
Validaciones
+
Tests
=
Funcionalidad terminada

97. Definition of Done general

Toda funcionalidad importante desarrollada a partir de este roadmap deberá cumplir al menos:

Backend

- modelo de dominio correcto;
- reglas centralizadas;
- validación;
- permisos;
- manejo de errores.

Runtime

- comportamiento real;
- estados válidos;
- recuperación ante decisiones manuales;
- consistencia.

Frontend

- opción visible solamente cuando funciona;
- mensajes comprensibles;
- feedback de estado;
- consistencia visual.

Testing

- caso normal;
- edge cases;
- valores mínimos;
- cantidades irregulares;
- estados inválidos.

Integración

- compatibilidad con Tournament Graph cuando corresponda;
 - compatibilidad con Competition Lab;
 - sin comportamientos diferentes entre Preview y Runtime.
-

98. Filosofía de desarrollo desde este punto

El proyecto deberá seguir principalmente cinco principios.

98.1. No engañar al usuario

Si algo no funciona:

se deshabilita

se oculta

o se marca Próximamente

98.2. Una sola fuente de verdad

Preview, Builder, Runtime y validadores no deben mantener reglas contradictorias.

98.3. Reutilización

Los motores deben compartir funcionalidades transversales.

Por ejemplo:

Series

Tie policies

Manual decisions
Routing
Result providers

no deberían reimplementarse independientemente cuatro veces.

98.4. Separación entre plantilla y ejecución

Debe distinguirse claramente:

qué puede ocurrir

de:

qué está ocurriendo ahora

La primera corresponde a plantillas.

La segunda corresponde al runtime.

98.5. Historia inmutable

Cuando una competición real comienza, los cambios futuros en las plantillas no deben cambiar retrospectivamente su historia.

99. Resumen ejecutivo del roadmap

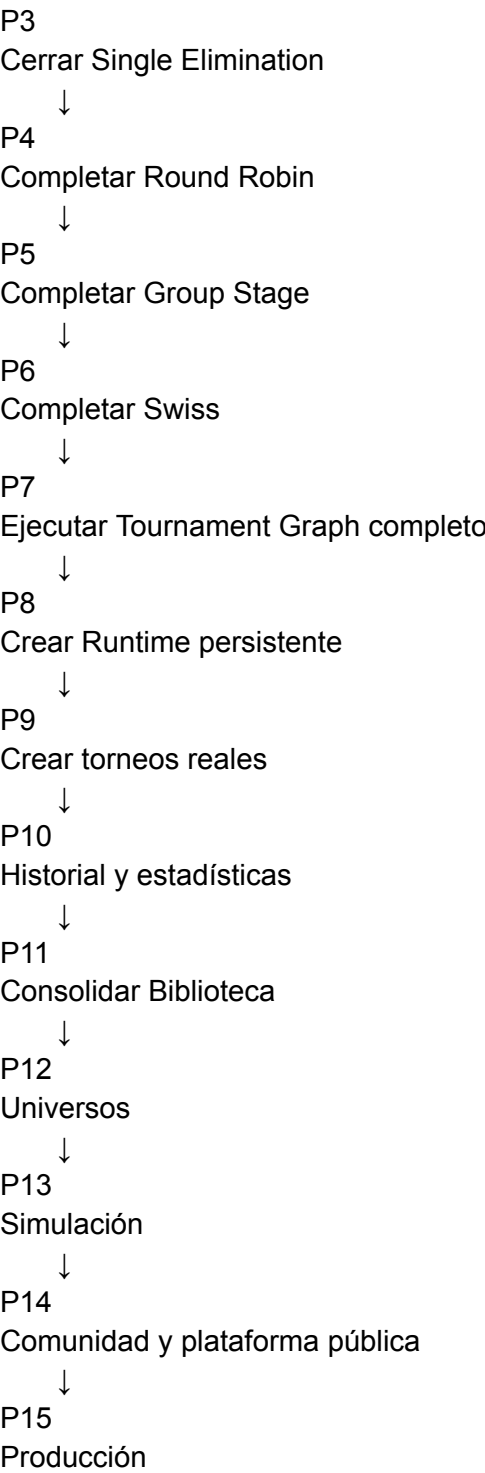
La situación actual puede resumirse de la siguiente manera.

OmniMerge ya cuenta con una base sólida de:

Biblioteca
Entidades
Atributos
Plantillas
Fases
Tournament Graph
Competition Lab

y Single Elimination ha alcanzado un grado de funcionalidad suficiente para convertirse en el primer motor de referencia.

El siguiente camino será:



100. Resumen de prioridades

Prioridad	Objetivo principal	Resultado
-----------	--------------------	-----------

P0	Evitar funcionalidades engañosas	UI y backend coherentes
P1	Igualar Preview y Runtime	Una misma lógica
P2	Funcionalidades avanzadas	Manual decisions, series, tie policies
P3	Cerrar Single Elimination	Engine Stable V1
P4	Completar Round Robin	Liga ejecutable completa
P5	Completar Group Stage	Grupos y clasificación avanzados
P6	Completar Swiss	Swiss dinámico real
P7	Completar Tournament Graph	Torneos multifase ejecutables
P8	Runtime persistente	Torneos reanudables
P9	Torneos reales	Entidades reales participando
P10	Historial y estadísticas	Historia permanente
P11	Consolidar Biblioteca	Reutilización completa
P12	Universos	Contexto y organización
P13	Simulación	Resultados automáticos
P14	Plataforma pública	Colaboración y publicación
P15	Producción	Plataforma estable

101. Visión final

La meta no consiste únicamente en construir una aplicación para crear brackets.

OmniMerge deberá convertirse en una plataforma donde puedan definirse:

qué existe,
 qué características posee,
 cómo se relaciona,
 en qué contexto existe,
 cómo interactúa,
 cómo compite,
 qué ocurrió,
 y cómo evoluciona.

Los torneos constituyen uno de los mecanismos mediante los cuales las entidades interactúan, pero no necesariamente serán el único mecanismo.

La arquitectura construida alrededor de entidades, versiones, atributos, colecciones, fases, grafos, runtime y futuras simulaciones debe permitir que OmniMerge evolucione sin quedar limitado a un único tipo de contenido o competición.

Por esta razón, la estrategia actual prioriza completar primero el **núcleo competitivo**, convertir Single Elimination en referencia, elevar los restantes motores al mismo estándar y construir posteriormente una ejecución persistente.

Una vez alcanzado ese punto, Biblioteca, Universos y Simulación podrán conectarse sobre una base estable.

La evolución prevista puede resumirse finalmente como:

DATOS
↓
ENTIDADES
↓
CONTEXTO
↓
COMPETICIÓN
↓
EJECUCIÓN
↓
RESULTADOS
↓
HISTORIA
↓
SIMULACIÓN
↓
UNIVERSO

Ese será el camino de desarrollo de OmniMerge a partir de su estado actual.